

Série des exercices poo

Exercice 1

1) classe Voiture

Ecrire une **classe Python** nommée **Voiture** ayant les méthodes et attributs suivants :

1. **marque** : (type string ie chaîne de caractères)
2. **puissance_fiscale** : (type int ie un entier)
3. **carburant** : (type string ie une chaîne de caractères)
4. **Créer un constructeur __init__** : constructeur de la classe.
5. **Créer une méthode afficher_infos()** : permettant d'afficher les informations de la voiture (marque, puissance fiscale, carburant).

2) héritage

Créez une **classe Renault** héritant de la classe **Voiture**.

Créer un attribut supplémentaire à la classe Renault : **modele** (type string ie une chaîne de caractères)

Créer un constructeur à la classe Renault : permettant d'appeler le constructeur de la **classe mère (Voiture)** et initialise l'**attribut modele**.

Ajoutez une méthode `afficher_modele()` : à la classe Renault qui affiche le modèle de la voiture.

3) Getters et Setters

Ajouter des méthodes **get_<attribut>** et **set_<attribut>** pour chaque attribut, permettant d'accéder et de modifier les attributs pour les **deux classes "Voiture" et "Renault"**.

4) Tester l'Héritage

1. Créez un **objet d'instance** de la **classe Renault**.
2. Se servir des **méthodes** pour **afficher les informations** et le **modèle de la voiture**, puis modifiez quelques **attributs** en utilisant les **setters**.
3. Réaffichez les **informations** pour confirmer que les modifications ont été prises en compte.

Exercice 2

Imaginez que vous travailliez sur le **développement d'un logiciel de gestion de véhicules** pour une entreprise de **location de véhicules**. On suppose qu'il existe deux types **principaux de véhicules** : les **voitures** et les **motos**. Votre objectif est de créer un système permettant de suivre et de gérer les informations relatives à chaque véhicule, notamment **la marque, le modèle et l'année de fabrication**. En outre, vous souhaitez que chaque véhicule puisse être **mis en marche** et **arrêté** à la demande.

Pour atteindre cet objectif, vous êtes obligé de suivre les étapes suivantes :

1. **Établissez une classe de base nommée "Vehicule"** : avec des attributs tels que "marque", "modele" et "annee". Cette classe doit être dotée de deux méthodes essentielles : "demarrer()" et "arreter()".

2. **Créez deux sous-classes, "Voiture" et "Moto"** : qui héritent des caractéristiques de la classe "Vehicule". Dans ces sous-classes, ajoutez des attributs spécifiques supplémentaires tels que "nombre_de_portes" pour les voitures et "cylindree" pour les motos.
3. **Personnalisez la méthode "init()"** : dans les sous-classes pour initialiser les attributs spécifiques de chaque type de véhicule en plus des attributs hérités de la classe "Vehicule".
4. **Modifiez les méthodes "demarrer()" et "arreter()"** : dans les sous-classes pour qu'elles affichent des messages appropriés, par exemple, "La voiture démarre" ou "La moto s'arrête", en fonction du type de véhicule.
5. **Créez des instances de voitures et de motos** : en utilisant les sous-classes "Voiture" et "Moto". Appelez ensuite les méthodes de ces instances pour mettre les véhicules en marche et les arrêter, tout en affichant les détails spécifiques à chaque véhicule.

Exercice 3

1) - Ecrire une **classe Python** nommée "Person" ayant comme attributs : **name** (nom), **age** (âge) et **gender** (genre). Implémentez ensuite les méthodes suivantes :

1. **Un constructeur __init__()** : qui prend les paramètres name, age et gender, et initialise les attributs correspondants de l'instance.
2. **Une méthode d'instance nommée introduce()** : qui affiche une présentation de la personne sous le format "Je m'appelle [name], j'ai [age] ans et je suis [gender]."

2) - Créez une autre **classe Python** nommée "Student" qui hérite de la **classe "Person"**. La **classe "Student"** aura un attribut supplémentaire : **section**. Implémentez les méthodes suivantes :

1. **Une méthode constructeur __init__()** : qui prend comme paramètres name, age, gender et section, et initialise les attributs correspondants de l'instance en utilisant la méthode d'initialisation de la classe parent "Person".
2. **Une méthode affiche une présentation de l'étudiant nommée introduce_student()** : qui affiche une introduction du type "Je m'appelle [name], j'ai [age] ans et je suis [gender]. Je suis dans la section [section]."